

פרויקט: Nexus Server — שרת HTTP

ב-C++20

מסמך: תיעוד הנדסי וארכיטקטורה | מחבר: אוראל שושן

תוכן עניינים

- פרק 1: תשתית הרשת ומחזור חיי ה-TCP Server
 - פרק 2: ניתוח בקשות HTTP וטיפול בקלט
 - פרק 3: מנוע המקביליות וניהול חוטים (ThreadPool)
 - פרק 4: ניתוב, אבטחה וקבצים סטטיים
 - פרק 5: ביצועים ובדיקות עומסים (k6)
- סיכום הנדסי ומבט-על (Engineering Executive Summary)

1. תשתית הרשת ומחזור חיי ה-TCP Server

1.1 סקירת מחלקת HttpServer

תפקיד המחלקה: המחלקה מהווה את נקודת הכניסה והניהול הראשית של השרת. היא אחראית על אתחול תשתית התקשורת והסוקטים (`setup_socket`), ניהול לולאת ההאזנה והקליטה של לקוחות חדשים תוך תמיכה בכיבוי מבוקר (`start` מבוסס הדגל `_is_running`), וניתוב הטיפול בכל לקוח (`handle_client`) לביצוע בחוט נפרד דרך ה-`ThreadPool`.

שדות ומאפייני המחלקה:

תיאור	שם השדה (Type)
מספר הפורט (שער התקשורת) ברמת המערכת שעליו הסוקט יאזין לבקשות HTTP נכנסות.	<code>port_ (uint16_t)</code>
ה- <code>File Descriptor</code> שמייצג את הסוקט המאזין של השרת מול מערכת ההפעלה. תפקידו הוא האזנה וקליטת בקשות חיבור בלבד (משויך לכתובת המקומית ול- <code>_port</code> , ולא ללקוח ספציפי). מאותחל כ-1 כדי לסמן שטרם נוצר.	<code>server_fd_ (int)</code>
משתנה דגל (<code>Flag</code>) המציין את מצב השרת. הוא מקבל <code>true</code> עם הפעלת השרת ב- <code>start()</code> , מחזיק את לולאת ההאזנה הראשית פעילה, ומשתנה ל- <code>false</code> בעת קריאה ל- <code>stop()</code> לצורך סגירה מבוקרת (<code>Graceful Shutdown</code>).	<code>is_running_ (bool)</code>
אובייקט בריכת חוטים המנהל תור משימות ומקצה חוט עבודה נפרד לכל לקוח חדש שנכנס. מנגנון זה מאפשר טיפול מקבילי (<code>Concurrent</code>) במספר לקוחות בו-זמנית ומונע מלקוח בודד לחסום את השרת כולו.	<code>thread_pool_ (WorkerThreadPool)</code>

1.2 מה זה File Descriptor (FD)?

File Descriptor (FD) הוא מספר שלם לא-שלילי (`int`) המשמש כמזהה ומתווך בין מרחב המשתמש (`User Space`) לבין הקרנל של מערכת ההפעלה.

- **אופן הפעולה מול הקרנל:** כאשר התוכנית יוצרת משאב חדש (כגון סוקט לתקשורת או קובץ בדיסק), הקרנל מקצה ומנהל את מבנה הנתונים המורכב של המשאב בתוך מרחב הזיכרון שלו (`Kernel Space`). במקום לחשוף מבנה פנימי זה, הקרנל מחזיר לתוכנית אינדקס מספרי מתוך טבלת המשאבים של התהליך (`File Descriptor Table`).
- **שימוש בקוד:** בכל פעולה עתידית שהשרת מבצע (האזנה, קריאה, כתיבה או סגירה של החיבור), התוכנית פונה לקרנל ומעבירה לו את אותו מספר שלם (`_server_fd` או `client_fd`), ומערכת ההפעלה יודעת בדיוק על איזה חיבור רשת פיזי לבצע את הפעולה.

- **עקרון "Everything is a file":** במערכות הפעלה מבוססות POSIX/Linux, כל פעולות הקלט/פלט (I/O) מטופלות באותו אופן אחיד דרך FDs, ללא תלות אם מדובר בקובץ מקומי בדיסק או בסוקט תקשורת ברשת.

1.3 מחזור חיי החיבור (TCP Lifecycle System Calls)

1.3.1 יצירת נקודת הקצה — socket()

קריאה לפונקציה זו מהווה פנייה ישירה לקרנל בבקשה להקצות נקודת קצה חדשה לתקשורת (Communication Endpoint).

בשלב זה:

- הקרנל מקצה זיכרון ומבנה נתונים פנימי לניהול החיבור.
- הקרנל מחזיר מזהה קובץ (File Descriptor) חדש מסוג int.
- ה-Socket עדיין אינו משויך לכתובת IP או למספר פורט ספציפיים.

פירוק הפרמטרים של הקריאה `socket(domain, type, protocol)`:

- **AF_INET (Address Family / Domain):**
 - מהות: הגדרת משפחת הכתובות ופרוטוקול הרשת שבהם נשתמש לתקשורת.
 - בפועל: מייצג כתובות רשת מסוג IPv4. לתמיכה ב-IPv6 נדרש להשתמש ב-AF_INET6.
- **SOCK_STREAM (Socket Type):**
 - מהות: הגדרת אופי התקשורת ואופן העברת המידע בשכבת התעבורה.
 - בפועל: מייצג זרם נתונים אמין מבוסס פרוטוקול TCP, הכולל:
- **חיבור מכוון (Connection-Oriented):** נדרשת לחיצת יד וביסוס קשר ראשוני טרם שליחת המידע.
- **אמינות (Reliability):** כל בית (Byte) מגיע ליעדו ללא אובדן נתונים ולפי סדר הזרימה המדויק.
- **0 (Protocol):**
 - מהות: בחירת פרוטוקול ספציפי מתוך משפחת הפרוטוקולים שנבחרה.
 - בפועל: העברת הערך 0 מורה לקרנל לבחור אוטומטית את פרוטוקול ברירת המחדל המתאים לשילוב של ה-Domain וה-Socket Type. עבור AF_INET ו-SOCK_STREAM, ברירת המחדל היא תמיד פרוטוקול TCP (IPPROTO_TCP).

1.3.2 שיוך כתובת ופורט — bind())

תפקידה של הפונקציה bind() הוא לקשור (לחבר) את ה-Socket שנוצר (_server_fd) לכתובת IP מקומית ולפורט ייעודי במחשב. ללא פעולה זו, הקרנל לא יידע לאיזה תהליך (Process) לנתב את חבילות הנתונים (TCP Packets) שמגיעות אל כרטיס הרשת.

הפרמטרים מוגדרים באמצעות המבנה sockaddr_in:

- **sin_family**: נקבע כ-AF_INET (משפחת כתובות IPv4).
- **sin_addr.s_addr** והשימוש ב-INADDR_ANY: הקבוע INADDR_ANY (השקול ל-0.0.0.0) מורה לקרנל להאזין לבקשות מכל ממשקי הרשת הזמינים במכונה (כרטיס רשת מקומי, חיבורי Ethernet/Wi-Fi ו-Localhost) ולא להגביל את השרת לכתובת IP בודדת.
- **סדר בתים והמרות באמצעות htons ו-htonl**
 - מעבדים מודרניים בארכיטקטורת x86/ARM שומרים נתונים בזיכרון בסדר בתים מסוג **Little Endian** (הבית הפחות משמעותי, LSB, נשמר בכתובת הזיכרון הנמוכה ביותר).
 - מנגד, פרוטוקולי הרשת (TCP/IP) מחייבים סדר בתים מסוג **Big Endian**, המוגדר כ-**Network Byte Order** (הבית המשמעותי ביותר נשלח ונשמר ראשון).
 - פונקציות ההמרה מבטיחות תאימות מול חומרת הרשת:
- **htons() (Host to Network Short)**: ממירה את מספר הפורט (משתנה בגודל 16 ביט).
- **htonl() (Host to Network Long)**: ממירה את כתובת ה-IP (משתנה בגודל 32 ביט).

1.3.3 מעבר להאזנה פסיבית — listen())

כברירת מחדל, סוקט שנוצר במערכת הוא סוקט אקטיבי המיועד ליזום פניות תקשורת כלפי חוץ. הקריאה לפונקציה listen()) משנה את מצב הסוקט (_server_fd) מאקטיבי לפסיבי — מעתה הוא אינו יוזם פניות, אלא ממתין לקבלת בקשות חיבור מלקוחות.

- **תהליך ה-TCP 3-Way Handshake ומבנה תור ה-Backlog:**

כאשר לקוח פונה לשרת, מתבצע מול מערכת ההפעלה תהליך לחיצת יד משולשת:

1. הלקוח שולח בקשת סנכרון (SYN).
 2. הקרנל משיב באישור ובקשת סנכרון משלו (SYN-ACK).
 3. הלקוח מאשר את הקבלה (ACK).
- עם סיום לחיצת היד ברמת הקרנל, החיבור מוכן ומועבר לתוך **תור ה-Backlog** — חדר המתנה מנוהל בזיכרון הקרנל, שבו החיבור ממתין עד שהשרת ישלוף אותו באמצעות הקריאה ל-accept().

- **הגדרת גודל התור באמצעות SOMAXCONN:**

הפרמטר השני בפונקציה מגדיר את קיבולת תור ההמתנה. בחירה בקבוע מערכת ההפעלה SOMAXCONN (Socket Maximum Connections) מורה לקרנל להקצות לתור זה את הגודל המקסימלי המותר ברמת הקונפיגורציה של מערכת ההפעלה (במקום ערך שרירותי קטן). הגדרה זו קריטית להתמודדות עם עומסי תעבורה גבוהים (High Concurrency) ומונעת דחיית חיבורים (Connection Refused) כשרצף פניות מהיר מגיע טרם שליפתן מהתור.

1.3.4 קבלת חיבור נכנס — accept ()

הפונקציה `accept ( )` אחראית על קליטת חיבורי לקוחות שהשלימו את לחיצת היד מול מערכת ההפעלה.

- **יעילות ומניעת שריפת מעבד (Blocking Mechanism):**
הקריאה לפונקציה היא חוסמת (Blocking). כל עוד תור ה-Backlog ריק מחיבורים מוכנים, הקרנל מרדים את החוט הראשי של השרת במקום להריץ לולאה אינסופית ששורפת משאבי מעבד (מניעת Busy-Waiting). ברגע שחיבור TCP חדש נכנס לתור, הקרנל מעיר את התהליך באופן מיידי.
- **שליפה מהתור והקצאת `client_fd`:**
כאשר חיבור זמין, `accept ( )` שולפת את הראשון בתור ה-Backlog ומחזירה **File Descriptor** חדש לגמרי (`client_fd`) המוקדש בלעדית לתקשורת מול אותו לקוח ספציפי, בעוד ה-`server_fd` ממשיך להאזין כרגיל לפניות נוספות בלולאה.
- **שמירת פרטי הלקוח ויציאה מבוקרת:**
הפונקציה מאכלסת את המבנה `sockaddr_in client_addr` בכתובת ה-IP והפורט של נקודת הקצה המרוחקת. במידה והשרת עובר כיבוי יזום דרך `stop ( )`, הסוקט הראשי נסגר, `accept ( )` מחזירה ערך שלילי, והבדיקה `break if (!is_running_)` מאפשרת יציאה נקייה מהלולאה (Graceful Shutdown).

1.4 מנגנון ההפרדה בין Listening Socket ל-Connected Socket

המערכת מבצעת הפרדה מבנית מוחלטת בין סוקט ההאזנה לבין סוקטי החיבור של הלקוחות:

- **סוקט האזנה (_server_fd):**
מזהה ייעודי שמשייך לכתובת ולפורט המקומיים בלבד. תפקידו פסיבי ומסתכם בהאזנה ובקליטת בקשות חיבור חדשות בלולאת השרת הראשית, ללא מעורבות בהעברת נתונים (Data Transfer).
- **סוקט חיבור (client_fd):**
מזהה ייחודי שנוצר על ידי הקרנל בעת הקריאה ל-accept(). סוקט זה מייצג חיבור מבוסס (Connected Socket) המוגדר ברמת הקרנל לפי רביעיית TCP מלאה: IP מקומי, פורט מקומי, IP של הלקוח ופורט של הלקוח. דרך סוקט זה מתבצעת התקשורת הבלעדית מול אותו לקוח (recv ו-send).
- **חשיבות ההפרדה ומניעת "זיהום משימות" (Data Corruption):**
הפרדה זו היא תנאי יסוד לפעולה תקינה של שרת מרובה-חוטים:
 1. **המשכיות השירות:** השרת ממשיך לקלוט לקוחות חדשים בחוט הראשי ללא חסימה בזמן שחוטי העבודה ב-ThreadPool מעבדים נתונים מול לקוחות קיימים.
 2. **בידוד נתונים:** אילו כל הלקוחות היו חולקים את אותו ה-FD, חוטים שונים היו קוראים וכותבים מאותו ערוץ, דבר שהיה גורם להשחתת מידע ("זיהום משימות") ולהחזרת תגובות של לקוח אחד ללקוח אחר לחלוטין.

1.5 הגדרת SO_REUSEADDR ומצב TIME_WAIT

בעת סגירה או אתחול מחדש של שרת TCP, מערכת ההפעלה מעבירה את החיבורים שנסגרו למצב השהיה מובנה בפרוטוקול הנקרא TIME_WAIT.

- **מהות מצב TIME_WAIT ברמת הקרנל:**
מערכת ההפעלה אינה משחררת את הפורט באופן מיידי, אלא שומרת אותו תפוס למשך פרק זמן קצוב (בדרך כלל כדקה עד שתי דקות). מנגנון זה נועד להבטיח שחבילות מידע ישנות שנתקעו ברשת (Delayed/Duplicate TCP Segments) יתפוגגו לחלוטין ולא יגיעו בטעות למופע החדש של השרת וישבשו את הנתונים שלו.
- **הבעיה ללא שימוש בדגל:**
במידה והשרת מופעל מחדש מיד לאחר כיבוי או קריסה, הקריאה לפונקציה bind () תיכשל מיד ותזרוק שגיאת מערכת מסוג EADDRINUSE (Address already in use), מכיוון שהפורט עדיין מסומן כתפוס בקרנל.
- **פתרון הבעיה באמצעות SO_REUSEADDR:**
העברת הדגל דרך הפונקציה setsockopt () מאפשרת לשרת לעקוף מגבלה זו. הדגל מורה לקרנל לאפשר שיוך (bind) מחדש של הפורט המקומי באופן מיידי, גם אם קיימים חיבורים קודמים הנמצאים כעת במצב TIME_WAIT. הגדרה זו הכרחית לאתחול מהיר, להמשכיות השירות (High Availability), ולסביבות פיתוח ובדיקות עומסים שבהן השרת מורם ומורד בתדירות גבוהה.

פרק 2: ניתוח בקשות HTTP וטיפול בקלט (Request Parsing)

2.1 פרוטוקול HTTP/1.1 ומבנה הבקשה

בקשת HTTP/1.1 מוגדרת כפרוטוקול טקסטואלי מובנה (Text-based Protocol) המורכב משלושה חלקים עיקריים:

- שורת הבקשה (Request Line):
 - השורה הראשונה בכל בקשה. שורה זו מחולקת לשלושה רכיבים המופרדים ביניהם על ידי רווח יחיד:
 - Method (מתודה): הפעולה שהלקוח מבקש לבצע (GET, POST, PUT, DELETE וכו').
 - Target / Path (נתיב): המשאב המבוקש בשרת (לדוגמה: index.html, /, /api/data).
 - HTTP Version (גרסה): גרסת הפרוטוקול שבה הלקוח משתמש (בשרת זה התמיכה ממוקדת ב-HTTP/1.1).
- כותרות הבקשה (Headers):
 - שורות מטא-דאטה המופיעות מיד לאחר שורת הבקשה, במבנה קבוע של מפתח וערך (Key: Value). הכותרות מספקות מידע חיוני על אופי הפנייה, כגון זהות המארח המבוקש (Host), פורמט הנתונים המועברים (Content-Type), וכמות הבתים המדויקת שנמצאת בגוף ההודעה (Content-Length).
- גוף הבקשה (Body / Payload):
 - התוכן הממשי של הבקשה (למשל אובייקט JSON, טקסט חופשי, או תוכן קובץ בינארי). ככלל, בבקשות מסוג GET גוף הבקשה אינו קיים (ריק), בעוד שבבקשות מסוג POST או PUT גוף ההודעה מכיל את הנתונים הנשלחים לעיבוד בשרת.

תפקיד ה-CRLF והמשמעות של הרצף הכפול:

- CRLF יחיד (r\n): על פי תקן HTTP/1.1 הרשמי (RFC 9112), כל סיום שורה בפרוטוקול חייב להסתיים ברצף התווים (r / 0x0D Carriage Return ו-n / 0x0A Line Feed).
- רצף CRLF כפול (r\n\r\n): התקן מחייב הפרדה מוחלטת בין בלוק הכותרות לבין גוף ההודעה באמצעות שורה ריקה. שורה ריקה זו מיוצגת על ידי שני רצפי CRLF עוקבים (r\n\r\n), המהווים את הגבול המפריד (Delimiter) שמאפשר ל-Parser לדעת במדויק היכן מסתיימת הכותרת והיכן מתחיל ה-Body.

2.2 קריאת הנתונים מהסוקט (recv) וניהול הזיכרון

הפונקציה `recv()` מבצעת קריאת מערכת (System Call) לקרנל, במטרה למשוך את זרם הבתים שהתקבלו ברשת מסוקט הלקוח (`client_fd`) ולהעתיק למאגר הזיכרון המקומי (`buffer`).

- משמעות ערכי ההחזרה של `bytes_read()` (`recv()`):
 - `bytes_read > 0`: הפעולה הצליחה; הערך מייצג את כמות הבתים המדויקת שנמשכה בפועל מהרשת והועתקה ל-Buffer.
 - `bytes_read == 0`: הלקוח יזם סגירה מסודרת של החיבור מצידו (/ Graceful Close / TCP FIN / EOF). במצב זה אין נתונים נוספים לקריאה ויש לשחרר את הסוקט.
 - `bytes_read < 0`: אירעה שגיאת תקשורת ברמת הרשת או בקריאת המערכת, הדורשת סגירה מיידית של החיבור.
- שיקולי הקצאת זיכרון ומבנה ה-Buffer:
 - גודל מאגר של `4096 (char buffer[4096])`: גודל זה נבחר בהתאמה לגודל דף זיכרון סטנדרטי במערכת ההפעלה (OS Page Size). קיבולת של 4096 בתים מאפשרת קריאה יעילה של מרבית בקשות ה-HTTP/1.1 הסטנדרטיות (כולל כלל הכותרות) בפעימת I/O אחת, ללא צורך בהקצאות זיכרון דינמיות יקרות ב-Heap.
- מניעת גלישת זיכרון והשמת `\0` (Null Terminator):
 - הגבלת הקריאה ל-`1 - sizeof(buffer)`: פרוטוקול TCP מעביר בתים בינאריים גולמיים (Raw Bytes) ואינו מצרף תו סיום מחרוזת. הפחתת בית אחד מגודל הקריאה מבטיחה שתמיד יישאר מקום פנוי של בית אחד לפחות בסוף המערך.
 - השמת `'\0' = buffer[bytes_read]`: השמה זו חותמת ידנית את סוף המידע הנקרא כמחרוזת C תקנית (Null-terminated String). פעולה זו קריטית לפני ההמרה ל-`std::string`: ללא ה-`\0`, בנאי המחרוזת ימשיך לקרוא זיכרון לא מאותחל מעבר לגבולות המערך (Buffer Over-read), דבר שיוביל להתנהגות לא מוגדרת (Undefined Behavior), לקריסת התוכנית או לחשיפת מידע רגיש מהזיכרון.

2.3 פיענוח הבקשה באמצעות HttpRequestParser

המחלקה HttpRequestParser אחראית על המרת מחרוזת ה-HTTP הגולמית (raw_request) למבנה נתונים מונחה-עצמים מסוג HttpRequest. התהליך מבוצע באופן סדרתי תוך שימוש ב-std::istream כדי להתייחס למחרוזת כאל זרם קלט (Input Stream):

- **חילוץ שורת הבקשה (Request Line):**

- הפונקציה `std::getline(stream, line)` שולפת את השורה הראשונה מתוך הזרם.
- השורה מועברת ל-Stream מקומי (`line_stream`), וממנה מחולצים שלושת השדות המרכזיים באמצעות אופרטור החילוץ (`<<`), אשר קורא מקטעים המופרדים ברווחים:
 - המילה הראשונה מושמת לשדה המתודה (`req.method`).
 - המילה השנייה מושמת לשדה התיב (`req.path`).
 - המילה השלישית מושמת לגרסת הפרוטוקול (`req.version`).

- **חילוץ כותרות (Headers Extraction) ומבנה מפתח/ערך:**

- לולאת `while (std::getline(stream, line))` מעבדת את שורות הכותרות העוקבות.
- **זיהוי סיום הכותרות:** הפונקציה בודקת קיום נקודתיים באמצעות `line.find(':')`. בהגעה לשורה המפרידה (השורה הריקה שבין הכותרות לגוף), לא נמצא תו נקודתיים והפונקציה מחזירה `std::string::npos`. מצב זה מהווה אינדיקציה לסיום אזור הכותרות והלולאה נשברת (`break`).
- **פירוק השורה:** המחרוזת נחתכת לשני חלקים:
 - המפתח נגזר מתחילת השורה ועד הנקודתיים (`line.substr(0, pos)`).
 - הערך נגזר החל מ-2 תווים לאחר הנקודתיים כדי לדלג על הרווח המפריד (`line.substr(pos + 2)`).
- **ניקוי תווי בקרה (`\r`):** מכיוון ש-`getline` מפצלת שורות לפי `\n`, התו `\r` (חלק מ-CRLF) נותר בסוף המחרוזת. הפקודה `value.pop_back()` מסירה תו זה ומבטיחה שהערך יישמר נקי.
- הנתונים נשמרים במבנה מפה אסוציאטיבית (`req.headers[key] = value`) המאפשר שליפה מהירה של כותרות לפי שמן.

- **חילוץ גוף הבקשה (Body):**

- כל המידע שנותר בזרם הקלט לאחר שבירת לולאת הכותרות משויך באופן בלעדי לגוף ההודעה.
- לולאה עוקבת שואבת את יתרת השורות ומשרשרת אותן לשדה `req.body`, תוך שמירה על מעברי השורות הנדרשים.

פרק 3: מנוע המקביליות וניהול חוטים (Worker Thread Pool)

3.1 ארכיטקטורת ה-ThreadPool ומבנה הנתונים

במערכות רשת בעלות עומס גבוה, קיימת עדיפות מובהקת לניהול מקבילי מבוסס בריכת חוטים מוגדרת מראש על פני מודל של הקצאת חוט ייעודי לכל בקשה (Thread-per-Client):

- **מניעת דלדול משאבים וקריסת מערכת (Resource Starvation):** במודל של פתיחת חוט לכל לקוח, קפיצה פתאומית בתעבורה (למשל אלפי בקשות בו-זמנית) תוביל להקצאת אלפי חוטים. מצב זה יכלה במהירות את משאבי הזיכרון של מערכת ההפעלה (הקצאת Call Stack נפרד של מספר מגה-בייטים לכל חוט) ויגרום לשגיאות דחיית חיבורים וקריסת התהליך.
- **צמצום תקורה ותופעת Context Switching:** יצירה והשמדה תכופה של חוטים דורשות קריאות מערכת יקרות. בנוסף, קיום של אלפי חוטים פעילים גורר עומס כבד של החלפת הקשר (Context Switching) במעבד, דבר שמבזבז מחזורי שעון יקרים. ה-ThreadPool פותר זאת על ידי יצירת מספר קבוע של חוטים בהתאם ליכולת העיבוד, אשר נשארים חיים לאורך כל פעולת השרת ומבצעים שימוש חוזר (Reuse) לטיפול במשימות מתוך התור.

משתני המחלקה ותפקידם בארכיטקטורה:

- `<workers_ (std::vector<std::thread`: מערך דינמי המאחסן ומנהל את מופעי חוטי העבודה של השרת. החוטים מאותחלים בבנאי ומריצים ברקע את הלולאה הפנימית (`worker_loop`).
- `<tasks_ (std::queue<std::function<void`: תור משימות מסוג `(FIFO (First In, First Out`). התור שומר משימות עבודה מופשטות הנארזות כפונקציות מסוג `std::function<void>()`, הממתינות לשליפה על ידי חוט פנוי.
- `<queue_mutex_ (std::mutex`: מנעול הדדי (Mutual Exclusion) שנועד להגן על תור המשימות (`_tasks`) מפני גישה מקבילית סימולטנית. המנעול מבטיח סנכרון ומונע תנאי מרוץ (Race Conditions) בעת הוספת משימות או שליפתן.
- `<cv_ (std::condition_variable`: משתנה תנאי המשמש לסנכרון ואיתות בין החוט הראשי שמייצר משימות לבין חוטי העבודה. מאפשר להרדים חוטים פנויים ביעילות כשהתור ריק, ולהעירם מיד עם כניסת משימה חדשה מבלי לבזבז זמן מעבד.
- `<stop_ (bool`: דגל בקרה בוליאני המציין האם הבריקה נמצאת בתהליך כיבוי. מאפשר לחוטי העבודה לזהות סיום פעילות ולצאת מלולאת העבודה באופן מסודר (Graceful Shutdown).

3.2 סנכרון תור המשימות והכנסת משימה (enqueue)

תור המשימות (`_tasks`) מהווה משאב משותף (Shared Resource) הקריטי לפעולת השרת: החוט הראשי של השרת מזין משימות חדשות באמצעות פעולת `push`, ובמקביל חוטי העבודה בבריכה שולפים משימות לביצוע באמצעות `pop`.

- **הגנה מפני תנאי מרוץ (Data Race Mitigation):**
גישה סימולטנית בלתי מתואמת של מספר חוטים לתור עלולה לשבש את המצביעים הפנימיים של מבנה הנתונים ואת מונה הגודל, ולהוביל להשחתת זיכרון ולקריסת התוכנית. השימוש במנעול הדדי מסוג `std::mutex` מבטיח גישה אקסקלוסיבית (Mutual Exclusion) ומאפשר רק לחוט בודד לקרוא או לעדכן את התור בכל רגע נתון.
- **ניהול הנעילה ועקרון RAII באמצעות תחום מוגדר ({}):**
הפונקציה עוטפת את מנגנון הנעילה בבלוק סגור באמצעות `std::unique_lock`:
 - **בנייה ושחרור אוטומטיים:** הבנאי של `unique_lock` נועל את ה-Mutex מיד עם יצירתו. ביציאה מהבלוק התחום ({}), האובייקט יוצא מתחום ההגדרה (Scope), מושמד, וה-Destructor שלו משחרר את המנעול באופן מיידי.
 - **בטיחות מול חריגות (Exception Safety):** עקרון ה-RAII מבטיח שגם אם תיזרק שגיאה בעת הוספת המשימה לתור, המנעול ישתחרר כחוק והשרת לא ייכנס למצב קיפאון (Deadlock).
 - **צמצום הקטע הקריטי (Minimal Critical Section):** תחום הנעילה מצומצם אך ורק לשורת ההכנסה לתור (`tasks_.push`), מה שמאפשר שחרור מהיר של המשאב ומונע עיכובים מיותרים של שאר החוטים.
- **איתות חסכוני ומניעת בעיית העדר השועט (Thundering Herd Problem):**
 - **ביצוע מחוץ לנעילה:** הקריאה ל-`cv_.notify_one()` מבוצעת רק לאחר סגירת הבלוק ושחרור ה-Mutex. הדבר מונע תופעת "Hurry-up and Wait", שבה חוט מתעורר אך נתקל במנעול תפוס ונאלץ להירדם שוב.
 - **בחירה ב-`notify_one()` על פני `notify_all()`:** בכל קריאה ל-`enqueue` נוספת משימה בודדת לתור. העברת אות התעוררות קולקטיבי (`notify_all`) הייתה גורמת לכלל חוטי הבריכה להתעורר, להתחרות על המנעול, כאשר רק חוט יחיד יזכה במשימה וכל השאר יגלו תור ריק וישובו להירדם. השימוש ב-`notify_one()` מעיר בדיוק חוט פנוי אחד, מונע החלפות הקשר (Context Switches) מיותרות, וחוסך משאבי מעבד קריטיים.

3.3 לולאת חוט העבודה (worker_loop) וניהול ההמתנה

כל חוט בברירת החוטים מריץ באופן עצמאי וממושך את הפונקציה worker_loop. הלולאה מבצעת סנכרון רציף בין מצב השינה של החוט לבין שליפת משימות וביצוען בפועל:

- **מנגנון ההמתנה היעיל של cv_.wait () ומניעת Busy-Waiting:**
במקום לבצע בדיקה מחזורית אינסופית הצורכת 100% ממשאבי המעבד, החוט מרדים את עצמו באמצעות משתנה התנאי.
 - בעת הקריאה ל-cv_.wait (lock, ...) מתבצעת פעולה אטומית כפולה: המנעול משתחרר כדי לאפשר לחוטים אחרים להמשיך לפעול, והחוט מוכנס למצב שינה יעיל בקרנל.
 - עם קבלת אות התעוררות (notify), החוט רוכש מחדש את המנעול (lock) באופן אוטומטי טרם המשך הריצה.
- **תפקיד ה-Predicate Lambda ומניעת התעוררויות שווא (Spurious Wakeups):**
במערכות הפעלה מודרניות, חוט עלול להתעורר מתרדמת עקב איתותי מערכת פנימיים גם ללא שליחת אות מכוונת מהקוד (Spurious Wakeup).
 - העברת פונקציית הלמבדא [] || return stop_ { } this() { }
tasks_.empty (!); { מבטיחה הגנה מלאה: החוט בודק את התנאי מיד עם התעוררותו.
 - במידה ומדובר בהתעוררות שווא (התור עדיין ריק והשרת לא בעצירה), החוט משחרר את המנעול ושב לישון באופן מיידי מבלי לנסות לגשת למשאב ריק ומבלי לייצר קריסות או התנהגות בלתי צפויה.
- **תנאי סיום וכיבוי מבוקר:**
הבדיקה return stop_ && tasks_.empty () if ; מאפשרת לחוט לזהות מצב של השבתת השרת. כאשר דגל העצירה מורם ולא נותרו משימות קודמות לעיבוד בתור, החוט מסיים את הלולאה ויוצא מהפונקציה בצורה חלקה ונקייה.
- **הפרדה בין שליפת המשימה להרצתה (Concurrency Preservation):**
 - **שליפה בתוך הנעילה:** שליפת המשימה (tasks_.front ()) והוצאתה מהתור (tasks_.pop ()) חייבות להתבצע תחת הגנת המנעול, מכיוון שתור המשימות הוא משאב משותף ורגיש לתנאי מרוץ (Data Races).
 - **ביצוע מחוץ לנעילה (task ()):** מיד עם שליפת המשימה, בלוק הסוגריים המסולסלים { } מסתיים והמנעול משתחרר. הרצת המשימה עצמה — הטיפול המלא בבקשת ה-HTTP של הלקוח — מתבצעת באופן מקבילי לחלוטין מחוץ לנעילה. אילו הפונקציה הייתה מורצת תחת המנעול, כל שאר עשרות החוטים בברירה היו קופאים וממתינים לסיום הטיפול בלקוח היחיד, דבר שהיה מבטל לחלוטין את מודל המקביליות והופך את השרת לסדרתי (Sequential).

3.4 השמדה וכיבוי מסודר (~WorkerThreadPool)

ההורס (Destructor) של המחלקה אחראי על תהליך כיבוי מבוקר (Graceful Shutdown) של כלל המשאבים בברירת החוטים, תוך הבטחה שאף חוט לא יישאר תקוע ושכל המשימות שבביצוע יושלמו:

- **שינוי הדגל וסנכרון זיכרון תחת נעילה:**
הדגל `_stop` מסומן כ-`true` בתוך בלוק תחום ומוגן על ידי `_queue_mutex`. פעולת הנעילה מבטיחה שכלל החוטים במערכת יראו את שינוי המצב באופן מסונכרן (Memory Visibility) מול בדיקת התנאי בלולאת העבודה שלהם.
- **איתות קולקטיבי באמצעות `cv_.notify_all()`:**
בניגוד להכנסת משימה בודדת שבה השתמשנו ב-`notify_one()`, בעת כיבוי השרת קיימת חובה להעיר את כל חוטי העבודה שישנים וממתינים למשימות. אילו היינו מעירים חוט בודד, שאר החוטים היו נותרים בתרדמת תמידית (Thread Leak / Deadlock), והשרת לעולם לא היה מצליח להשלים את תהליך הסגירה. האיתות הקולקטיבי גורם לכל החוטים להתעורר, לזהות שהדגל `stop_ == true` והתור ריק, ולצאת מהלולאה (`return`).
- **סנכרון סיום העבודה באמצעות `worker.join()`:**
ההורס עובר על מערך החוטים ומפעיל `worker.join()` על כל אחד מהם. קריאה זו היא פעולה חוסמת (Blocking), המשהה את החוט הראשי של ההורס עד שחוט העבודה הספציפי מסיים לחלוטין את פעולתו (כולל השלמת משימה שהייתה באמצע טיפול) ונסגר כחוק. תהליך זה מבטיח שלא נקטעים חיבורי לקוחות באופן חד ושהשרת משתחרר בצורה נקייה ומבוקרת.
- **בדיקת `worker.joinable()` ומניעת קריסת תוכנית (`std::terminate`):**
על פי תקן שפת ++C, הרס של אובייקט מסוג `std::thread` כאשר הוא עדיין מוגדר כ-`joinable` (כלומר מייצג חוט פעיל במערכת שטרם בוצע לו `join` או `detach`) מוביל לקריאה אוטומטית ובלתי הפיכה ל-`std::terminate()`, אשר מרסקת את התוכנית כולה באופן מיידי. הבדיקה מוודאת כי החוט אכן פעיל וזקוק לסנכרון, ומונעת התנהגות בלתי מוגדרת בעת סיום חיי האובייקט.

פרק 4: שרת הקבצים הסטטיים והגנת אבטחה (Static File Serving & Security)

4.1 מיפוי נתיבי URL למערכת הקבצים המקומית

שרת הרשת אחראי לתווך בין בקשות משאבים וירטואליות של הלקוח לבין קבצים פיזיים המאוחסנים בדיסק המקומי:

- **תרגום נתיב ה-req.path (HTTP) לנתיב דיסק מקומי:**
השרת מגדיר תיקיית בסיס ייעודית (Root Directory, כדוגמת /public) שרק מתוכה מותר לחשוף קבצים לרשת. בעת קבלת בקשה, השרת מחבר (Concatenates) את נתיב תיקיית הבסיס המקומית עם הנתיב המבוקש שהתקבל באובייקט הבקשה (req.path). לדוגמה, פנייה עבור assets/style.css/ תתורגם ישירות לנתיב הפיזי המקומי public/assets/style.css/.
- **טיפול בנתיב ברירת מחדל (Default Index Document):**
כאשר לקוח פונה לנתיב השורש הראשי של האתר (/), מערכת הקבצים אינה יכולה לקרוא ספרייה כקובץ תוכן. בהתאם לתקן שרתי Web סטנדרטיים, השרת מזהה פנייה זו ומנתב אותה באופן אוטומטי לקובץ ברירת המחדל: index.html. פעולה זו מתרגמת את הנתיב המבוקש ל-.public/index.html/, ובכך מאפשרת הגשת דף הבית בצורה שקופה ללקוח ללא צורך בציון שם הקובץ המפורש בכתובת ה-URL.

4.2 קריאת קבצים בינאריים וזיהוי סוגי מדיה (MIME Types)

הגשת תוכן סטטי לרשת מחייבת שמירה קפדנית על שלמות הנתונים והעברת מטא-דאטה מוגדר לדפדפן:

- **קריאה בינארית יעילה (`std::ios::binary`) ומניעת השחתת מידע:**
 - **הבעיה בקריאה טקסטואלית:** בפתיחת קבצים במצב טקסטואלי רגיל (ברירת המחדל של `std::ifstream`), מערכות הפעלה עשויות לבצע המרה אוטומטית של תווי ירידת שורה (כגון המרת רצף `\n` ל-`\n` ולהפך).
 - **ההכרח במצב בינארי:** בעת הגשת קובצי תמונה (`.jpg`, `.png`), סקריפטים, או קובצי בינאריים כגון `.wasm` (WebAssembly), כל בית בודד (Byte) הוא קריטי. תרגום אוטומטי של תווים ישחית את המבנה הפנימי של הקובץ (`Data Corruption`), יגרום לתמונות לא להיטען ולסקריפטים להיכשל בהרצה.
 - **הפתרון:** השימוש בדגל `std::ios::binary` מורה לספריית הקלט/פלט של `++C` לקרוא את זרם הבתים ישירות מהדיסק בדיוק כפי שהוא (Raw Byte-for-Byte), ללא שום מניפולציה או פענוח של תווים מיוחדים.
- **תפקיד ה-MIME Types וקותרת ה-Content-Type:**
 - **הסתמכות הדפדפן:** כאשר דפדפן מקבל זרם בתים מהשרת, הוא אינו מנחש את מהות הקובץ על פי שמו בלבד, אלא מסתמך באופן בלעדי על קותרת התשובה Content-Type (ה-MIME Type).
 - **אבטחה ותאימות:** אם השרת יחזיר משאב (כמו גיליון עיצוב CSS או סקריפט JS) עם קותרת שגויה (כגון `text/plain`), דפדפנים מודרניים יחסמו את הפעלת המשאב מטעמי אבטחה והגנה מפני התקפות (MIME-Sniffing Protection).
 - **ברירת המחדל:** עבור סיומות שאינן מוכרות לשרת, נעשה שימוש בסוג המדיה הכללי `application/octet-stream`, המורה לדפדפן להתייחס לתוכן כזרם בתים גולמי המיועד להורדה ולא לתצוגה ישירה.

4.3 הגנת אבטחה מפני מתקפת חציית נתיבים (Path Traversal Protection)

פתיחת שרת קבצים סטטיים לרשת ללא בקרת קלט עלולה לחשוף את מערכת ההפעלה המארכת לפגיעויות אבטחה קריטיות:

- **וקטור ההתקפה והסכנה בריצה ללא הגנה (Path / Directory Traversal):**
במערכות הפעלה מבוססות POSIX/Linux, הסימון המוסכם . . מייצג עלייה היררכית לתיקיית האב. תוקף פוטנציאלי עשוי לשלוח בקשת HTTP מניפולטיבית הכוללת רצף של תווים אלה (לדוגמה: GET ../../../../etc/passwd HTTP/1.1).
במידה והשרת משרשר נתיבים בצורה עיוורת, מערכת הקבצים של ה-OS תפתור את הנתיב ותאפשר בריחה מגבולות התיקייה הציבורית (/public), מה שיוביל לחשיפת קובצי מערכת רגישים (כגון קובצי משתמשים והרשאות, משתני סביבה, או קוד המקור של השרת עצמו).
- **מנגנון ההגנה ואימות הנתיב בקוד:**
 - **זיהוי מוקדם של תבנית חשודה:** טרם פנייה כלשהי למערכת הקבצים בדיסק (std::ifstream), השרת מבצע ולידציה של מחרוזת ה-req.path ובודק האם היא מכילה את הרצף . . באמצעות != req.path.find(" . . ")
std::string::npos.
 - **בלימה מידית ומניעת גישה:** ברגע שמזוהה רצף זה, השרת מסווג את הפנייה כחריגת אבטחה וקוטע את תהליך הטיפול באופן מיידי. השרת אינו ניגש לדיסק כלל, ובכך מונע כל אפשרות לעלייה בהיררכיית התיקיות.
 - **קוד תגובה מבוקר:** השרת מחזיר ללקוח תגובת שגיאה מסוג 403 Forbidden (או 404 Not Found), ובכך מסכל את ניסיון הפריצה ושומר על בידוד מוחלט של תיקיית השורש.

(k6) פרק 5: ביצועים ובדיקות עומסים

5.1 מתודולוגיית הבדיקה ומדדי ביצועים מרכזיים (KPIs)

על מנת לאמת את יציבות תשתית הרשת ואת יעילות מנוע המקביליות (WorkerThreadPool) שפותחו ב-C++, נערכו בדיקות ביצועים ועומסים סינתטיות באמצעות כלי הבדיקה התעשייתי k6:

- חשיבות השימוש בכלי k6:
הכלי k6 הינו כלי מודרני מבוסס Go המאפשר לדמות תעבורת רשת מקבילית בעומסים כבדים על ידי שימוש במשתמשים וירטואליים (Virtual Users - VUs). בדיקה זו היוותה את מבחן הקצה האמיתי לתקינות המערכת: היא אימתה כי מנגנון הסנכרון באמצעות `std::mutex` ו-`std::condition_variable` עומד במעמסה ללא נעילות הדדיות (Deadlocks), ללא גלישות זיכרון, ותוך חלוקת עבודה יעילה ורציפה בין כלל החוטים בבריכה.
- מדדי המפתח שנבחנו (KPIs):
 - תפוקת עבודה (Throughput / RPS): כמות הבקשות המלאות מקצה לקצה שהשרת מסוגל לעבד, להגיש מהדיסק ולהחזיר למשתמש בכל שנייה (Requests per Second).
 - פרופיל זמני השהיה (Latency Profile): משך הזמן הכולל מעת שליחת הבקשה על ידי הלקוח ועד לקבלת התגובה המלאה. נמדדו ערכי ממוצע, חציון (Median), ורף אחוזון 95 המייצג את זמני המענה עבור 95% מהבקשות.
 - שיעור הצלחה ונשירת חיבורים (Success & Drop Rate): אחוז החיבורים שנכשלו, נותקו או חוו שגיאת שרת תחת עומס. יעד הבדיקה הוגדר כ-0% שגיאות.

5.2 תרחיש הבדיקה וניתוח התוצאות בפועל

השרת נבדק תחת תנאי עומס רציפים בסביבה מקומית למשך 2 דקות רצופות, תוך הפעלת 100 משתמשים וירטואליים מקביליים (Concurrent VUs 100).

משמעות הנדסית	תוצאה בפועל	מדד ביצועי (Metric)
נפח תעבורה כולל ורציף לאורך 2 דקות ללא דעיכת ביצועים	+62,737 בקשות	סה"כ בקשות שעובדו
0 חיבורים שנשרו, 0 כשלי I/O או תקלות שרת תחת עומס	100.00%	שיעור הצלחה (Success Rate)
קצב עיבוד יציב של מעל 520 בקשות שלמות בשנייה	כ-522.4 RPS	קצב תפוקה (Throughput)
זמני תגובה מהירים הכוללים קריאת קובץ מהדיסק והגשתו ברשת	6.86 מילי-שניות	זמן השהיה ממוצע (Avg Latency)
מחצית מכלל הבקשות נענו תוך כ-5 מילי-שניות בלבד	5.29 מילי-שניות	זמן השהיה חציוני (Median Latency)
מענה יציב ומהיר גם עבור הבקשות בקצה טווח העומס	17.24 מילי-שניות	רף אחוזון 95 (p_{95}) Latency

מסקנות הנדסיות מניתוח התוצאות:

- חוסן מנוע המקביליות: השגת 100% הצלחה תחת 100 VUs מקביליים מוכיחה כי תכנון ה-WorkerThreadPool מנע בהצלחה תופעות כגון Thundering Herd ודלדול משאבים (Resource Starvation), ואפשר ניצול אופטימלי ומאוזן של משאבי העיבוד.
- יציבות שרת הקבצים והזיכרון: שרת הקבצים הסטטיים ביצע קריאה ושינוע של עשרות אלפי קבצים במצב בינארי ללא דליפות זיכרון (Memory Leaks), ללא השחתת נתונים, ותוך שמירה על זמני תגובה עקביים של מילי-שניות בודדות.

סיכום הנדסי ומבט-על (Engineering)

(Executive Summary)

בפרויקט זה תכננתי ומימשתי מאפס (Zero-Dependency) שרת HTTP/1.1 עתיר ביצועים בסביבת Linux/POSIX, תוך שימוש ביכולות המודרניות של שפת C++20. מטרתי המרכזית הייתה להשיג שליטה בלתי-אמצעית בתשתיות הליבה של מערכות הפעלה ותקשורת מחשבים: ניהול File Descriptors, מניפולציות זיכרון ברמת הבתים הבודדים, סנכרון רב-חוטי מורכב ואבטחת נתונים קפדנית.

אינטגרציית שכבות המערכת

את ארכיטקטורת השרת ביססתי על צימוד מדויק ויעיל בין ארבע שכבות ליבה:

- **שכבת התקשורת והרשת (Network & POSIX Layer):**
מימשתי מחזור חיים מלא של TCP Socket באמצעות קריאות המערכת הקלאסיות (socket, setsockopt, bind, listen, accept). יצרתי הפרדה מבנית בין ה-Listening Socket הקבוע לבין ה-Connected Sockets שנפתחים עבור כל לקוח בנפרד, ובכך הבטחתי האזנה רציפה ובלתי מופרעת לתעבורה נכנסת. בנוסף, הגדרתי את הדגל SO_REUSEADDR כדי לאפשר שחרור וקשירה מהירה מחדש של הפורט ללא עיכובי TIME_WAIT.
- **שכבת הפענוח והזרמת הנתונים (Parsing & I/O Layer):**
ביצעתי קריאת בתים גולמיים מהקרנל באמצעות מאגר מקומי מוגדר של recv (4KB), והקפדתי על סגירת מחרוזת בטוחה בעזרת \0 כדי למנוע גלישות זיכרון ותופעות של Buffer Over-read. פיתחתי את מחלקת ה-HttpRequestParser לביצוע פירוק שיטתי של הפרוטוקול הטקסטואלי בהתאם לתקן RFC 9112: חילצתי בצורה אוטומטית את שורת הבקשה (Request Line), זיהיתי כותרות לפי מבנה מפתח וערך, והסתמכתי על גבול ה-CRLF הכפול (\r\n\r\n) כדי ליצור חציצה נקייה בין כותרות הבקשה לבין גוף ההודעה (Body).
- **מנוע המקביליות והסנכרון (Concurrency & ThreadPool Layer):**
תכננתי ופיתחתי פתרון מקבילי מבוסס תור משימות (Producer-Consumer) ובריכת חוטים קבועה מראש (WorkerThreadPool). באמצעות מודל זה מנעתי קריסות הנובעות מדלדול משאבים (Resource Starvation) ותקורה מיותרת של יצירת חוטים ו-Context Switching. יישמתי מנגנון סנכרון קפדני לפי עקרון ה-RAII בעזרת std::unique_lock, השתמשתי בהתעוררות חסכונית באמצעות cv_.notify_one(), ומנעתי התעוררויות שווא (Spurious Wakeups) בעזרת Predicate Lambda. בנוסף, הפרדתי לחלוטין את הקטע הקריטי: שליפת המשימה מתבצעת תחת מנעול, בעוד שעיבוד הבקשה עצמה מתבצע במקביליות מלאה מחוץ לנעילה. ב-Destructor מימשתי כיבוי מבוקר (Graceful Shutdown) הממתין לסיום החוטים ומונע שגיאות של std::terminate או ניתוקים פתאומיים.
- **שכבת ההגשה והאבטחה (Static Serving & Hardening):**
יישמתי קריאה בינארית של משאבים מהדיסק (std::ios::binary) במטרה לשמור על שלמות קובצי תמונה, קוד וסקריפטים, והגדרתי מיפוי והצמדה של כותרת Content-Type (ערכי MIME Types) תקינים כדי למנוע חסימות בדפדפנים מודרניים. כמו כן, הטמעתי מנגנון אימות נתיב קשיח המזהה וחוסם ניסיונות בריחה מספרית (Path Traversal באמצעות ..), ובכך בודדתי לחלוטין את מערכת ההפעלה המארכת מפני חשיפת קבצים רגישים.

תוצאות והוכחת ביצועים

כדי לאמת את חוסן המערכת, הרצתי מבחני עומס וביצועים סינתטיים באמצעות כלי הבדיקה התעשייתי k6, שסיפקו אימות מספרי מובהק לציבות הארכיטקטורה שבנית:

- **עמידות תחת עומס רציף:** השרת עיבד בהצלחה למעלה מ-62,700 בקשות לאורך 2 דקות רצופות מול 100 משתמשים וירטואליים מקביליים (VUs).
- **אמינות מלאה:** השגתי 100.00% הצלחה ללא אף חיבור שנשר (dropped connections 0), ללא כשלי I/O וללא תקיעות מערכת או נעילות הדדיות (Deadlocks).
- **זמני תגובה תחרותיים:** שמרתי על תפוקה יציבה של מעל 522 בקשות לשנייה (RPS), עם זמן השהיה חציוני זריז של 5.29 מילי-שניות ורף של 17.24 מילי-שניות באחוזון ה-95 (\$p_{95}\$).

פרויקט זה ממחיש הלכה למעשה כיצד שילוב בין הבנה מעמיקה בעקרונות ה-Low-Level של מערכת ההפעלה לבין תכנות מודרני, בטוח ומדויק ב-C++, אפשר לי לבנות תשתית שרתים מהירה, יעילה, יציבה ומאובטחת לחלוטין.